

Analysis of SQL Injection and Cross-Site Scripting (XSS) Vulnerabilities Using Burp Suite and OWASP Juice Shop

Project Report

Submitted By

Ibtesum Arif

Introduction

Web applications are widely used in modern systems such as banking, shopping, and communication platforms. Due to their widespread usage, they are also common targets for cyberattacks. Two major web vulnerabilities are SQL Injection (SQLi) and Cross-Site Scripting (XSS).

SQL Injection occurs when attackers manipulate database queries through user input, which may lead to authentication bypass or unauthorized access. XSS occurs when malicious scripts are injected into web pages and executed in users' browsers.

This project analyzes these vulnerabilities using Burp Suite and OWASP Juice Shop in a controlled environment.

Objectives

The objectives of this project are:

- To set up OWASP Juice Shop and Burp Suite
- To analyze SQL Injection vulnerabilities
- To analyze Cross-Site Scripting vulnerabilities
- To understand how manipulated input affects application behavior
- To discuss mitigation techniques

Tools Used

OWASP Juice Shop

OWASP Juice Shop is an intentionally vulnerable web application designed for security learning and testing.

Burp Suite Community Edition

Burp Suite is a web security testing tool used to intercept and modify HTTP requests between the browser and server.

Installation and Setup

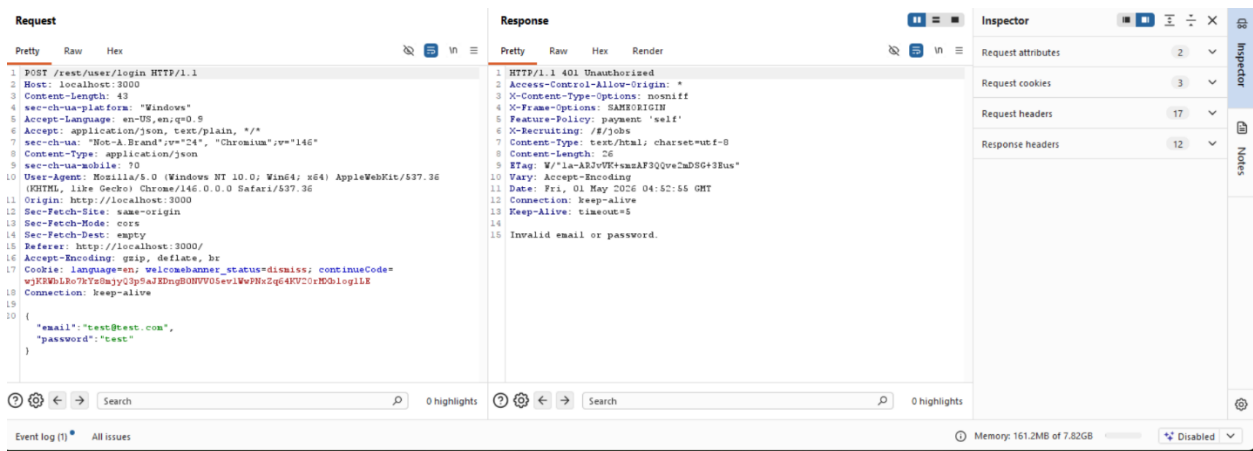
OWASP Juice Shop was installed locally on localhost port 3000.

Burp Suite Community Edition was configured as a proxy tool so that all browser traffic passed through it. This allowed HTTP requests and responses to be inspected and modified during testing.

Testing and Analysis

Normal Login Attempt

A normal login attempt was first performed using random credentials. The server responded with an “invalid email or password” message, confirming normal authentication behavior.



HTTP request and response for failed login

SQL Injection Testing

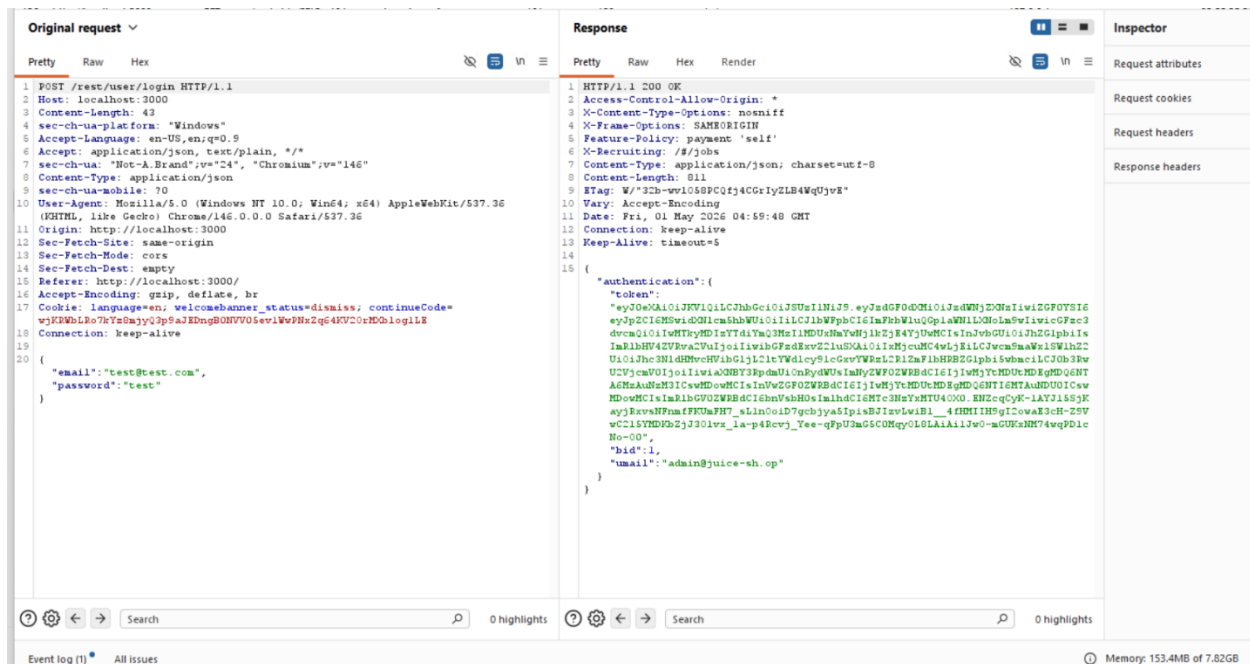
The login request was intercepted using Burp Suite. Input fields were modified with crafted input to test for SQL Injection.



Modified request in HTTP history

After forwarding the modified request, the application successfully authenticated the user and returned an authentication token.

This confirmed that the application was vulnerable to SQL Injection because authentication could be bypassed through manipulated input.



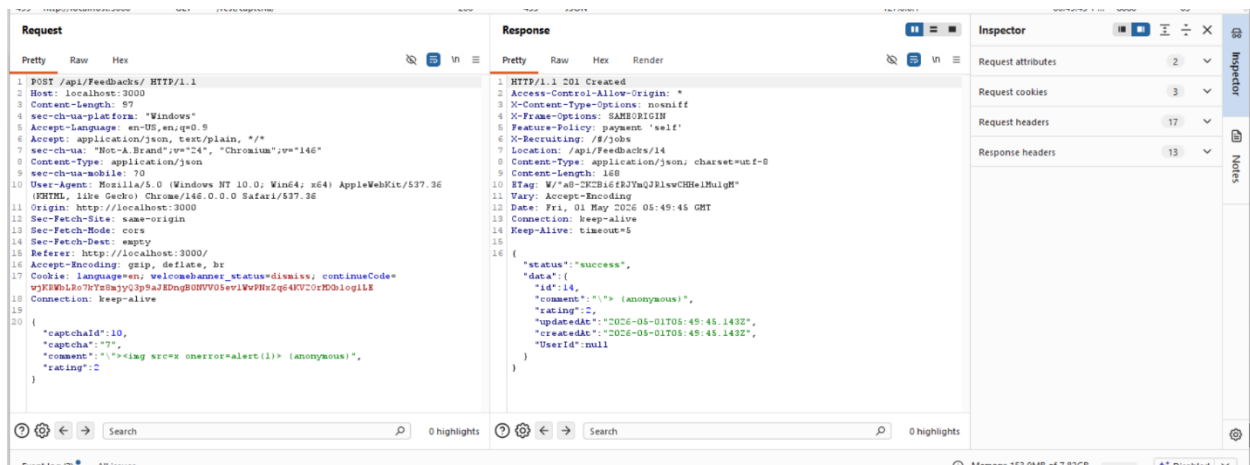
Successful authentication response

Cross-Site Scripting (XSS) Testing

For XSS testing, script-like input was submitted into review and feedback fields.

The application accepted and stored the input successfully. However, no script execution was observed in the browser.

This indicates that the application may have partial protection mechanisms such as output encoding or filtering.



XSS input was accepted

Findings

The following findings were observed during testing:

- SQL Injection vulnerability was confirmed through authentication bypass
- Weak input validation exists in the application
- XSS input was accepted but not executed
- The system demonstrates security weaknesses in handling user input

Mitigation Techniques

The following security practices can help prevent SQL Injection and XSS vulnerabilities:

- Proper input validation
- Using parameterized queries instead of dynamic SQL
- Output encoding before displaying user input
- Implementing Content Security Policy (CSP)

These techniques help reduce the risk of web application attacks.

Limitations and Future Work

This project used Burp Suite Community Edition, which has limited features compared to the Professional version.

Advanced features such as automated scanning and deeper vulnerability analysis were not available.

Future work may include testing additional vulnerabilities and using advanced security testing tools.

Conclusion

This project successfully analyzed SQL Injection and Cross-Site Scripting vulnerabilities using Burp Suite and OWASP Juice Shop.

SQL Injection testing demonstrated authentication bypass through manipulated input, while XSS testing showed weak handling of script-like input.

The project provided practical understanding of web security vulnerabilities and highlighted the importance of secure coding practices.